

CMS Credicitrus — Banco de Dados e Integração

Orientação operacional para provisionamento do SQL Server, execução dos scripts e devolutiva dos dados necessários à integração com a aplicação Next.js.

Versão: 1.2 · Data: 24/09/2026 · Ambiente alvo: Dev e/ou Produção (isolados)

1. Objetivo

Preparar um banco SQL Server dedicado ao CMS do novo site Credicitrus, com schema, categorias iniciais e usuário administrador, e devolver à equipe de desenvolvimento os parâmetros de conexão e infraestrutura para integração no projeto.

2. Pré-requisitos (lado TI)

- Instância Microsoft SQL Server acessível pela aplicação (rede/firewall liberados).
- Permissão para criar database e login (ou database já criada + usuário com DML/DDL necessários).
- Pasta de uploads **fora** do diretório sobrescrito no deploy, com escrita para a conta do processo Node.
- Scripts SQL entregues pela equipe de desenvolvimento na pasta `sql/` do repositório.

3. Estrutura do banco

Tabela	Função
DocumentCategory	Categorias por módulo (relatórios, normativos, ética, tarifas, assembleia)
Document	Documentos/PDFs/links publicados no site
HeroCampaign	Banners da home (desktop/mobile + CTA)
CmsUser	Usuários do painel /admin
CmsUserPermission	Permissão por módulo (none view edit publish)
CmsAuditLog	Auditoria de ações do CMS

Coluna de segurança: `CmsUser.SessionVersion` — invalida sessões após troca de senha.

4. Procedimento — criação do banco e scripts

1. **Criar o database** (um por ambiente), por exemplo:

```
CREATE DATABASE CredicitrusCms;  
GO  
USE CredicitrusCms;  
GO
```

Em produção, usar nome acordado e registrar em `DATABASE_NAME` .

2. **Criar login/usuário SQL** com senha forte, permissões no database (SELECT/INSERT/UPDATE/DELETE nas tabelas do CMS; evitar sa em aplicação).
3. **Executar os scripts nesta ordem exata** (SSMS, Azure Data Studio ou sqlcmd), conectado ao database criado:
 1. `001_schema.sql` — tabelas de conteúdo + audit
 2. `002_seed_categories.sql` — categorias iniciais
 3. `003_users.sql` — usuários e permissões
 4. `004_session_version.sql` — SessionVersion
 5. `005_seed_admin_suave.sql` — usuário inicial super-admin

Os scripts são idempotentes (podem ser reexecutados com segurança).

4. **Validar** com consultas rápidas:

```
SELECT name FROM sys.tables ORDER BY name;
SELECT Username, DisplayName, IsSuperAdmin, IsActive FROM dbo.CmsUser;
SELECT Module, Slug, Title FROM dbo.DocumentCategory ORDER BY Module, SortOrder;
```

5. **Criar pastas de upload** (fora do path de deploy):

```
{CMS_UPLOAD_PATH}/
relatorios/
normativos/
etica/
tarifas/
assembleia/
banners/
```

Exemplos Windows: `D:\dados\credicitrus\prod\uploads`

Exemplos Linux: `/var/dados/credicitrus/uploads`

6. **Testar conectividade** a partir do servidor da aplicação (host, porta 1433 ou a configurada, TLS conforme política).

5. Usuário inicial do CMS

Campo	Valor
Painel	/admin/login
Login (Username)	suave@suave.ppg.br
Nome de exibição	Suave
Perfil	Super-admin (acesso total)
Senha inicial	Su!5ttazSVDmakaqdpnE

O CMS autentica por **Username** (não há coluna de e-mail). O endereço acima foi cadastrado como login.

Obrigatório: trocar a senha no primeiro acesso. Não reutilizar a senha inicial como senha definitiva de produção.

Produção: o login bootstrap via variáveis `ADMIN_USERNAME` / `ADMIN_PASSWORD` do `.env` está **desabilitado**. Use somente usuários da tabela `CmsUser`.

6. Variáveis de ambiente — formato e devolutiva

Obrigatório: se a TI preencher as variáveis no Azure DevOps, no servidor ou em qualquer painel de configuração, os **mesmos valores preenchidos** devem ser enviados depois à equipe de desenvolvimento (canal seguro), para atualização / validação no projeto e alinhamento do deploy. Não basta configurar "lá": sem a devolutiva, o código e o pipeline do time de desenvolvimento ficam desatualizados.

Preferencialmente **um bloco por ambiente** (Dev e Prod separados). Nunca versionar senhas ou segredos no Git.

6.1 Formato de cada variável

Use exatamente os nomes abaixo. Valores booleanos são literais `true` ou `false` (minúsculos, sem aspas). Não use espaços em branco no início/fim do valor.

Variável	Obrig.	Formato / valores aceitos	Exemplo
<code>NODE_ENV</code>	Sim	Texto fixo: <code>production</code> , <code>development</code> ou <code>test</code> . Em Dev/Prod do servidor use <code>production</code> se o build for de produção.	<code>production</code>

Variável	Obrig.	Formato / valores aceitos	Exemplo
NEXT_PUBLIC_SITE_URL	Sim	URL absoluta pública, com protocolo <code>https://</code> , sem barra final.	<code>https://www.credicitrus.com.br</code>
CMS_ENABLED	Sim	Booleano. Só ativa o CMS se for exatamente <code>true</code> . Qualquer outro valor = CMS desligado (fallback estático).	<code>true</code>
DATABASE_HOST	Sim	Hostname DNS ou endereço IPv4/IPv6 acessível pelo servidor da aplicação. Sem protocolo e sem porta.	<code>sql-credicitrus.interno</code>
DATABASE_PORT	Sim*	Número inteiro (porta TCP). Se omitido, a aplicação assume <code>1433</code> .	<code>1433</code>
DATABASE_NAME	Sim	Nome do database SQL Server (identificador; sem espaços preferencialmente).	<code>CredicitrusCms</code>
DATABASE_USER	Sim	Login SQL da aplicação (texto). Não usar <code>sa</code> em produção.	<code>credicitrus_cms_app</code>
DATABASE_PASSWORD	Sim	Senha do login SQL (texto livre; caracteres especiais permitidos). Enviar por canal seguro.	<i>(senha forte gerada pela TI)</i>
DATABASE_ENCRYPT	Sim*	Booleano. TLS na conexão. Se omitido ou diferente de <code>false</code> , a app trata como criptografado (<code>true</code>).	<code>true</code>
DATABASE_TRUST_SERVER_CERTIFICATE	Sim*	Booleano. Só <code>true</code> confia no certificado do servidor (útil em lab/self-signed). Em produção preferir <code>false</code> + certificado válido.	<code>false</code>
CMS_UPLOAD_PATH	Sim	Caminho absoluto no disco do servidor da app, fora do diretório sobrescrito no deploy. Windows: usar barras invertidas ou escapadas conforme o painel; Linux: path POSIX.	<code>D:\dados\credicitrus\prod\uploads</code> <code>/var/dados/credicitrus/uploads</code>
CMS_MEDIA_PUBLIC_PREFIX	Não	Prefixo de URL pública dos arquivos, começando com <code>/</code> , sem barra final. Padrão se omitido: <code>/media</code> .	<code>/media</code>
ADMIN_AUTH_SECRET	Sim	Segredo opaco, mínimo 32 caracteres aleatórios (hex ou base64). Gerar com <code>openssl rand -hex 32</code> . Único por ambiente; rotacionar no go-live se necessário.	<code>a1b2c3...</code> (64 hex)

Variável	Obrig.	Formato / valores aceitos	Exemplo
TRUST_PROXY	Cond.	Booleano. Use <code>true</code> atrás de reverse proxy (IIS, nginx, Azure) para confiar em <code>X-Forwarded-For</code> . Em local sem proxy: omitir ou <code>false</code> .	<code>true</code>
SITE_ROBOTS_INDEX	Não	Booleano opcional. <code>true</code> = forçar indexação; <code>false</code> = forçar noindex. Se omitido: indexa quando o gate de acesso estiver desligado.	<code>true</code>
SITE_ACCESS_ENABLED	Não	Booleano. Gate temporário /acesso. Manter <code>false</code> em produção pública. Só <code>true</code> ativa o gate.	<code>false</code>
SITE_ACCESS_PASSWORD	Cond.	Texto. Obrigatório apenas se <code>SITE_ACCESS_ENABLED=true</code> .	(se gate ativo)
SITE_ACCESS_SECRET	Cond.	Segredo ≥ 32 caracteres (ex.: <code>openssl rand -hex 32</code>). Obrigatório se o gate estiver ativo.	(se gate ativo)
ADMIN_USERNAME / ADMIN_PASSWORD	Não (prod)	Bootstrap via .env. Em <code>NODE_ENV=production</code> está bloqueado . Não usar em Prod; login só via <code>CmsUser</code> .	—

* "Sim*" = fortemente recomendado enviar o valor explícito mesmo quando há padrão na aplicação.

6.2 Confirmações operacionais (além das variáveis)

- ☐ Scripts SQL 001–005 executados com sucesso no database informado
- ☐ Subpastas de upload criadas e permissão de escrita da conta do Node
- ☐ Política de backup da pasta `CMS_UPLOAD_PATH`
- ☐ Firewall/VPN: servidor da app consegue alcançar `DATABASE_HOST:DATABASE_PORT`
- ☐ HTTPS / reverse proxy confirmados (e se `TRUST_PROXY` foi aplicado)

6.3 Modelo de resposta (preencher e devolver)

Copiar o bloco abaixo, preencher com os valores reais do ambiente e enviar à equipe de desenvolvimento para atualização no projeto / pipeline.

```
Ambiente: [Dev | Prod]
Data do preenchimento:

NODE_ENV=production
NEXT_PUBLIC_SITE_URL=https://
CMS_ENABLED=true

DATABASE_HOST=
DATABASE_PORT=1433
DATABASE_NAME=CredicitrusCms
DATABASE_USER=
DATABASE_PASSWORD=
DATABASE_ENCRYPT=true
DATABASE_TRUST_SERVER_CERTIFICATE=false

CMS_UPLOAD_PATH=
CMS_MEDIA_PUBLIC_PREFIX=/media

ADMIN_AUTH_SECRET=
TRUST_PROXY=true
SITE_ROBOTS_INDEX=true
SITE_ACCESS_ENABLED=false

Scripts 001-005: [OK | pendente]
Pastas upload: [OK | pendente]
Observações / firewall / VPN / proxy:
```

7. Passo a passo — deploy no Azure

A aplicação é Next.js (Node). O pipeline e a infraestrutura ficam a cargo da TI. Não há `azure-pipelines.yml` no repositório: configure o pipeline no Azure DevOps conforme o padrão interno.

7.1 Ordem recomendada

1. **SQL Server** — criar database, usuário e executar scripts 001–005 (seção 4).
2. **Storage de uploads** — provisionar pasta persistente (seção 7.2).
3. **App Service (ou equivalente)** — criar o app Node por ambiente (Dev / Prod).
4. **Variáveis** — preencher App Settings / Variable Group / Key Vault (seção 6).
5. **Pipeline** — build e publicação (seção 7.4).
6. **Validação** — checklist pós-deploy (seção 7.5).
7. **Devolutiva** — enviar bloco 6.3 preenchido à equipe de desenvolvimento.

7.2 Uploads persistentes (obrigatório)

No Azure App Service, o sistema de arquivos do site é **efêmero**: um novo deploy ou reinício pode apagar arquivos gravados na pasta do app. Os uploads do CMS (`CMS_UPLOAD_PATH`) **não** podem ficar nesse disco.

- Preferir **Azure Files** montado no App Service (path de mount estável).

- Apontar `CMS_UPLOAD_PATH` para o caminho montado (ex.: `/home/site/wwwroot-data/uploads` ou o path Windows equivalente no mount).
- Garantir permissão de escrita da identidade do processo Node e backup da share.
- Criar (ou deixar a app criar) as subpastas: `relatorios`, `normativos`, `etica`, `tarifas`, `assembleia`, `banners`.

7.3 App Service / runtime

Item	Orientação
Runtime	Node.js LTS compatível com o projeto (verificar <code>package.json</code> / <code>engines</code> se houver)
Build	<code>npm ci → npm run build</code>
Start	<code>npm start</code> (equivale a <code>next start</code>)
HTTPS	Terminar TLS no App Service / Front Door / Application Gateway; redirecionar HTTP→HTTPS
Proxy	Com Front Door / gateway na frente: <code>TRUST_PROXY=true</code>
Ambientes	Dev e Prod isolados (apps, Variable Groups, SQL e share de upload separados)

7.4 Pipeline Azure DevOps (modelo)

1. Conectar o repositório Git e apontar o branch de release (ex.: `main`).
2. Criar Variable Group por ambiente (ou App Settings no App Service) com as variáveis da seção 6 — **secrets** em Key Vault / variáveis secretas do pipeline.
3. Estágio de build (agente Linux ou Windows, conforme padrão TI):

```
npm ci
npm run build
```

4. Publicar artefato (pasta do app após build, incluindo `.next`, `package.json`, `node_modules` de produção ou reinstalar no release com `npm ci --omit=dev`).
5. Estágio de deploy: publicar no App Service do ambiente alvo e injetar as App Settings (não gravar senhas no YAML do repositório).
6. Confirmar que o processo sobe com `npm start` e que as variáveis estão visíveis ao Node em runtime (não só em build, exceto `NEXT_PUBLIC_*`).

Importante sobre `NEXT_PUBLIC_SITE_URL` : prefixo `NEXT_PUBLIC_` é embutido no bundle no momento do *build*. Defina o valor correto no pipeline de build de cada ambiente (ou build separado por ambiente).

7.5 Checklist pós-deploy

- ☐ Site público abre (home) sem erro 500
- ☐ `/admin/login` carrega e autentica com o usuário inicial (seção 5)

- ❑ Upload de PDF de teste em um módulo (ex.: Relatórios) e arquivo aparece em `CMS_UPLOAD_PATH`
- ❑ URL pública do arquivo via prefixo `/media/...` responde 200
- ❑ Páginas de Transparência listam o item publicado
- ❑ Após novo deploy, o arquivo de upload **ainda existe** (prova de storage persistente)
- ❑ `SITE_ACCESS_ENABLED=false` em produção pública
- ❑ Troca da senha do super-admin no primeiro acesso

7.6 Rede e SQL

- App Service (ou outbound IPs) deve alcançar `DATABASE_HOST:DATABASE_PORT`.
- Se SQL for Azure SQL / SQL em VNet: configurar firewall / private endpoint conforme política.
- `DATABASE_ENCRYPT=true` em produção; `DATABASE_TRUST_SERVER_CERTIFICATE=false` com certificado válido.

8. Responsabilidades

TI	Desenvolvimento
SQL, scripts, storage persistente, App Service, pipeline, HTTPS/proxy, backups, variáveis no Azure	Código no Git, <code>.env.example</code> , validação funcional do CMS após o deploy
Preencher variáveis e devolver o bloco 6.3 com os valores usados	Alinhar documentação/projeto com a devolutiva e apoiar testes de publicação

9. Contato / próximos passos

1. TI executa banco e storage (seções 4, 5 e 7.2).
2. TI configura App Service + variáveis + pipeline (seções 6 e 7).
3. TI envia o bloco preenchido (6.3) por canal seguro à equipe de desenvolvimento.
4. Validar checklist 7.5 e trocar a senha de `suave@suave.ppg.br`.

Documento gerado para handoff TI · Projeto Credicitrus (Next.js + SQL Server CMS).

Scripts: pasta `sql/` · Variáveis: `.env.example` · Senha inicial do admin: somente neste PDF (não versionar em README).